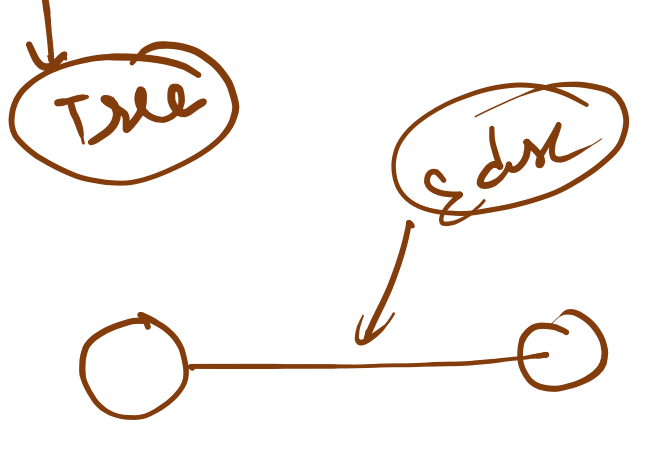
Lec-37

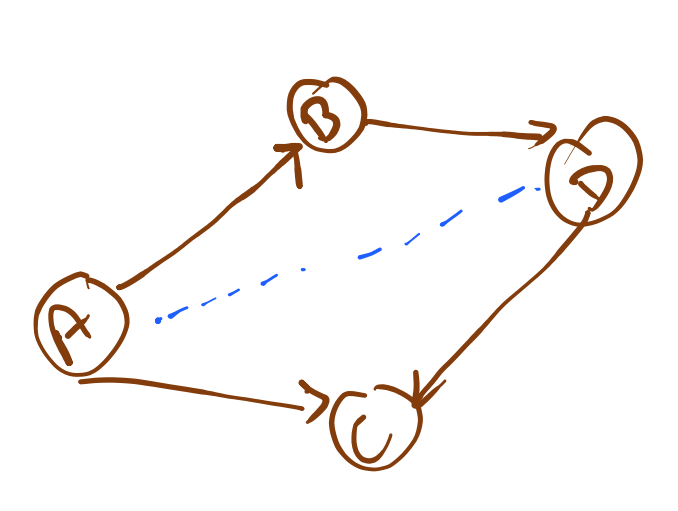
12/07/26 9:59 AM

① Graph → Tree → Graph

Graph → Tree → Graph → Tree



def ÷ this is the finite Edge and V-tx

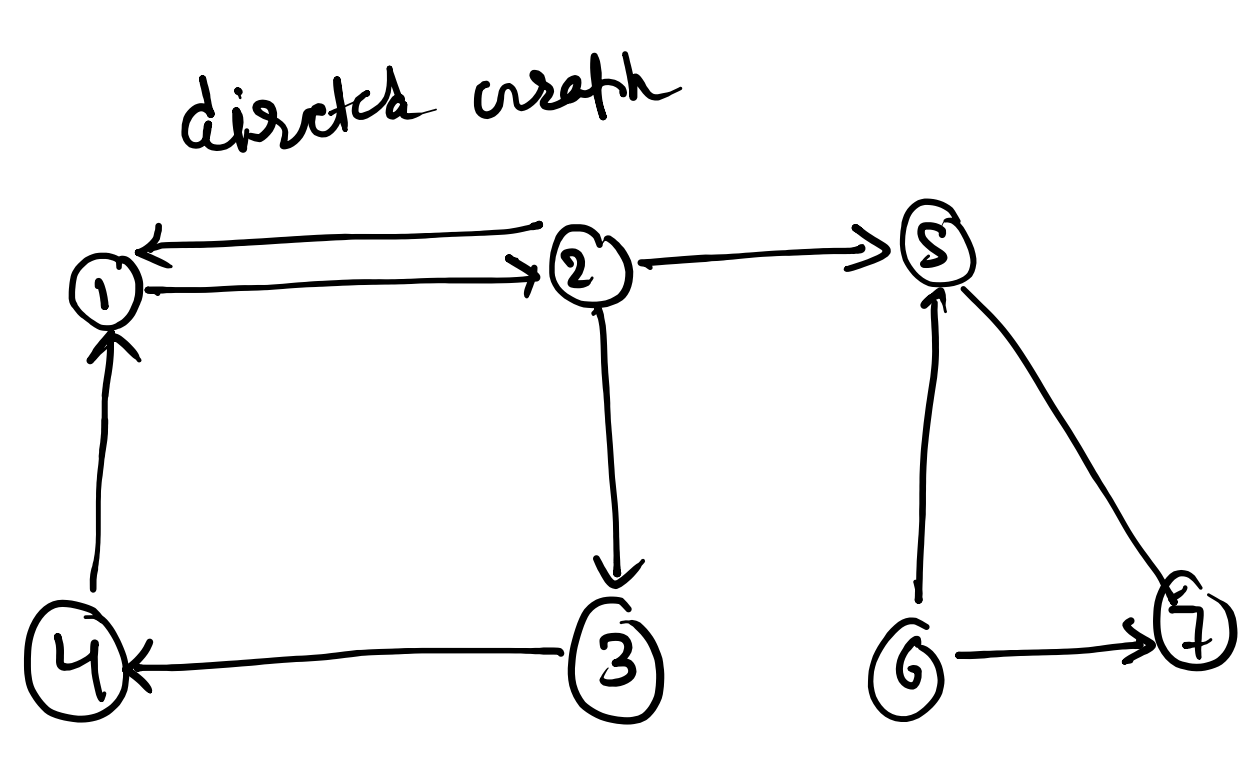


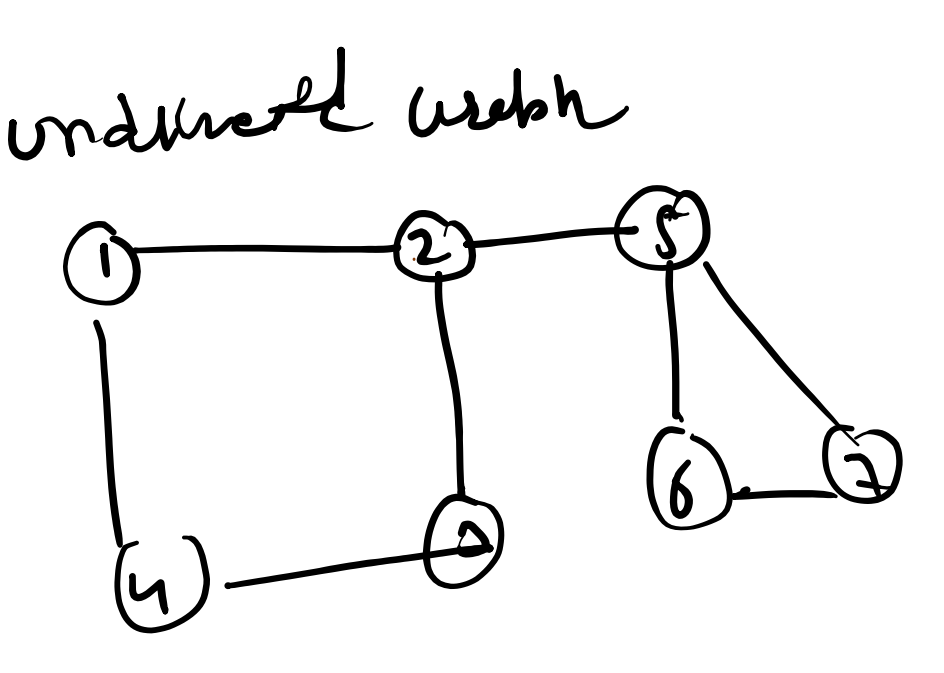
Use → Router → Routing table

Map → (A) time B (AI)

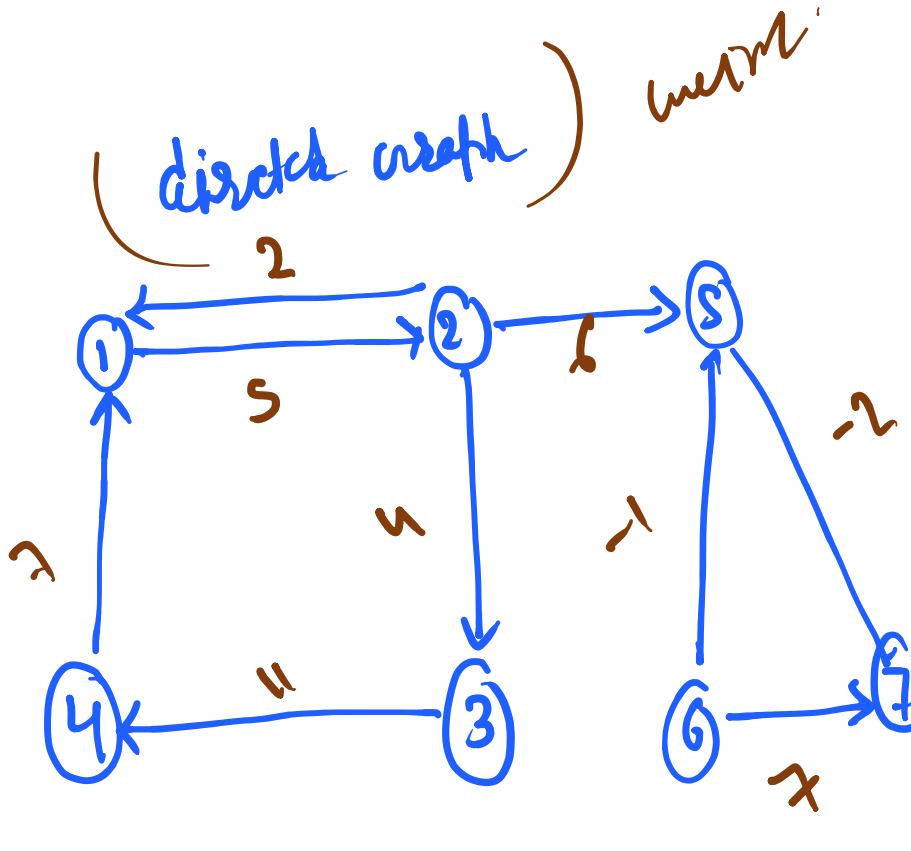
→ Safe path

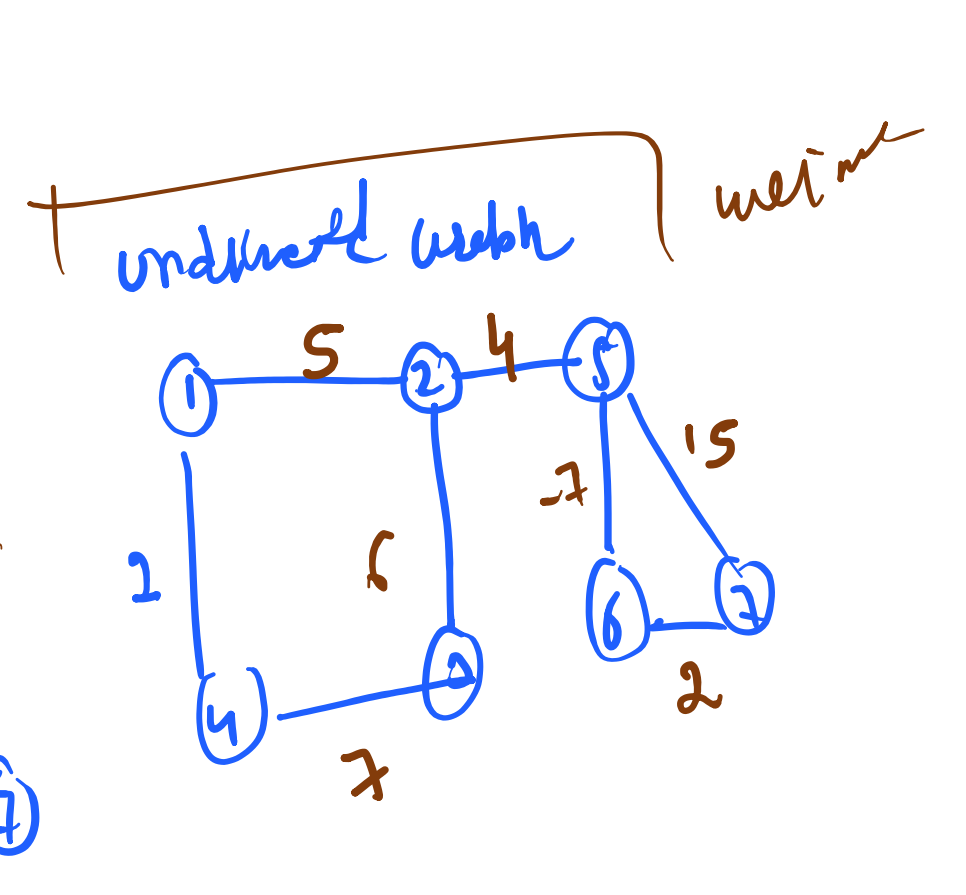
①

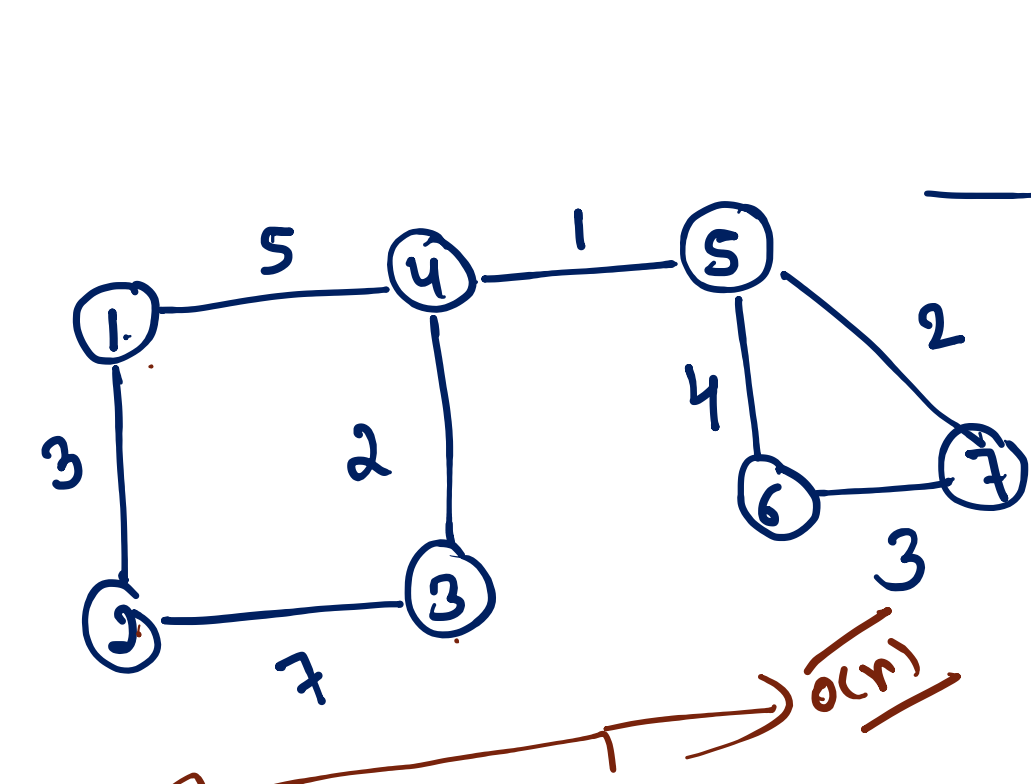

directed graph
1
2
3
4
5
6
7


undirected graph
1
2
3
4
5
6
7

① weighted
② unweighted


directed graph
weight
2
5
6
-2
7
4
-1
11
7


undirected graph
weight
5
4
15
2
6
-7
2
7



→ Adj Matrix O(1)

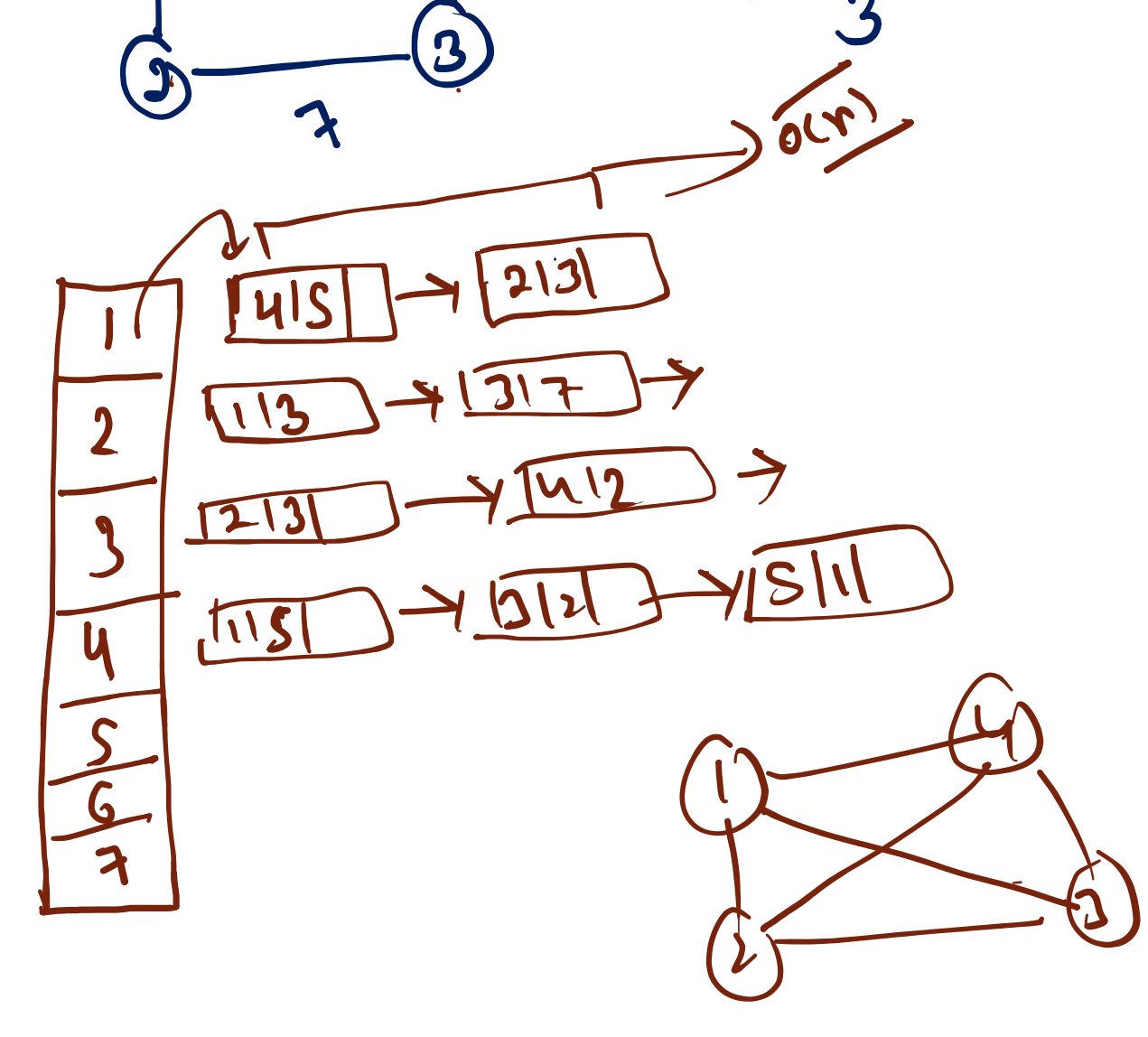
	1	2	3	4	5	6	7
1	0	3	0	5	0	0	0
2	3	0	7	0	0	0	0
3	0	7	0	2	0	0	0
4	5	0	2	0	1	0	0
5	0	0	0	1	0	4	2
6	0	0	0	0	4	0	3
7	0	0	0	0	2	3	0

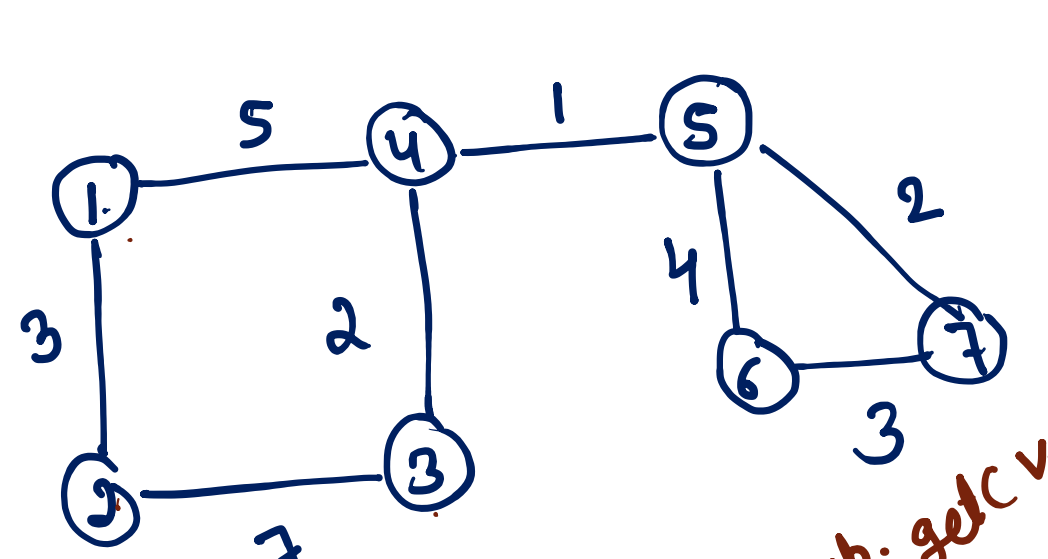
7×7

n → ① → $\overline{n-1}$

$\frac{n\times(n-1)}{2}$

nC_2


O(n)
1
2
3
4
5
6
7
4|5
2|3
1|3
3|7
2|3
4|2
1|5
3|2
5|1



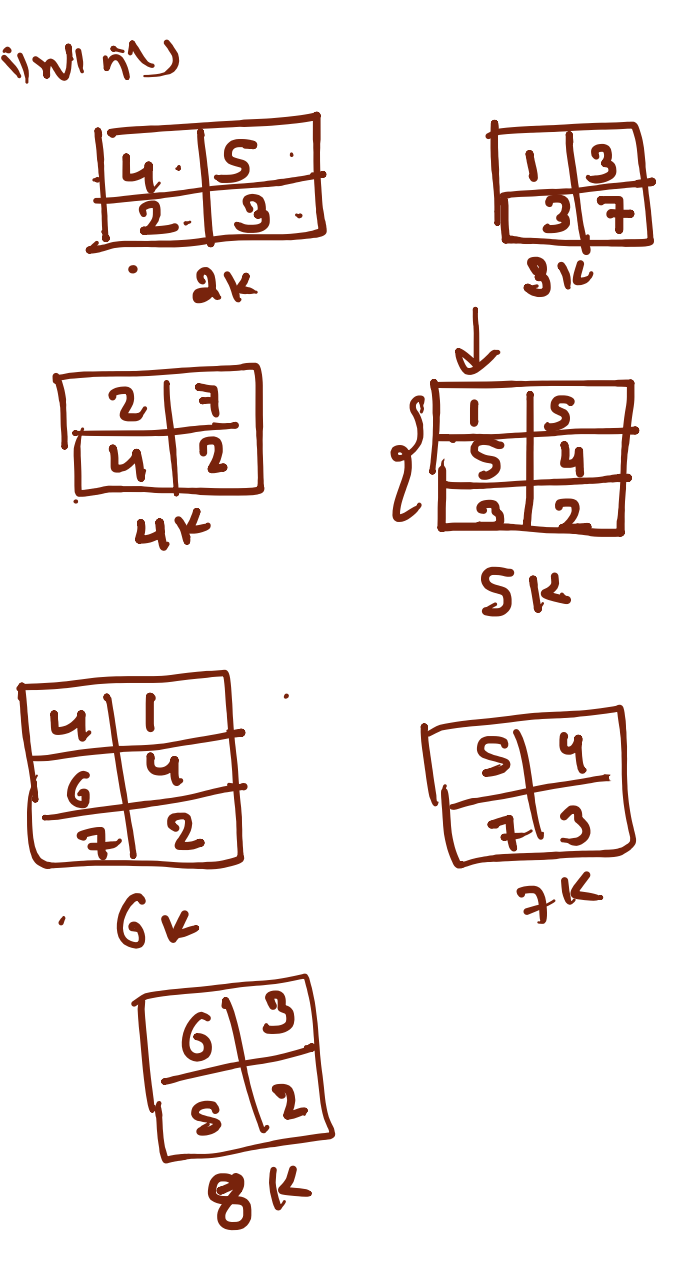
(1,5,3)

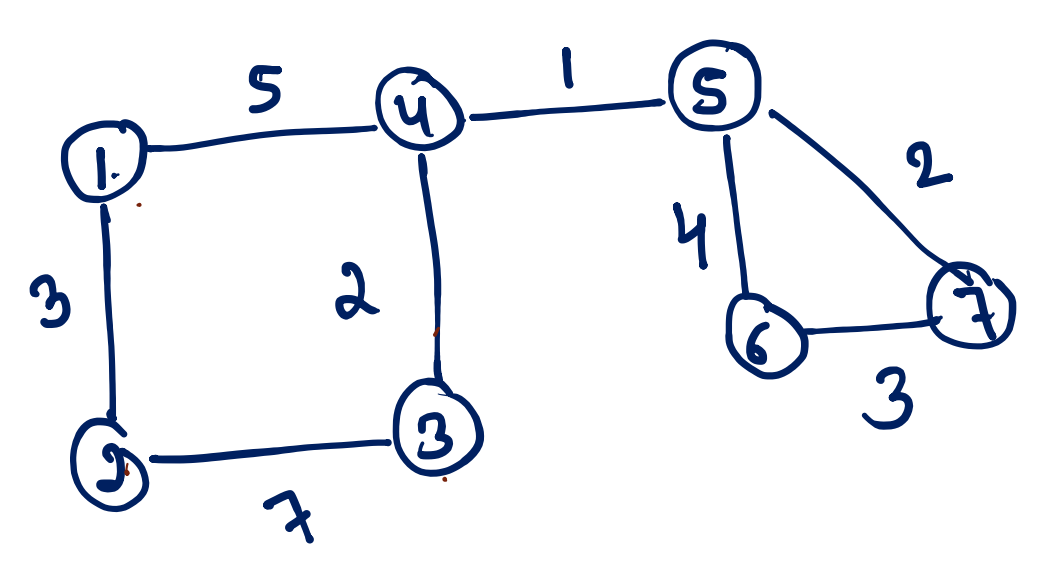
for(int nbrs: map.get(v).keySet(){
map.get(nbrs).remove(v)

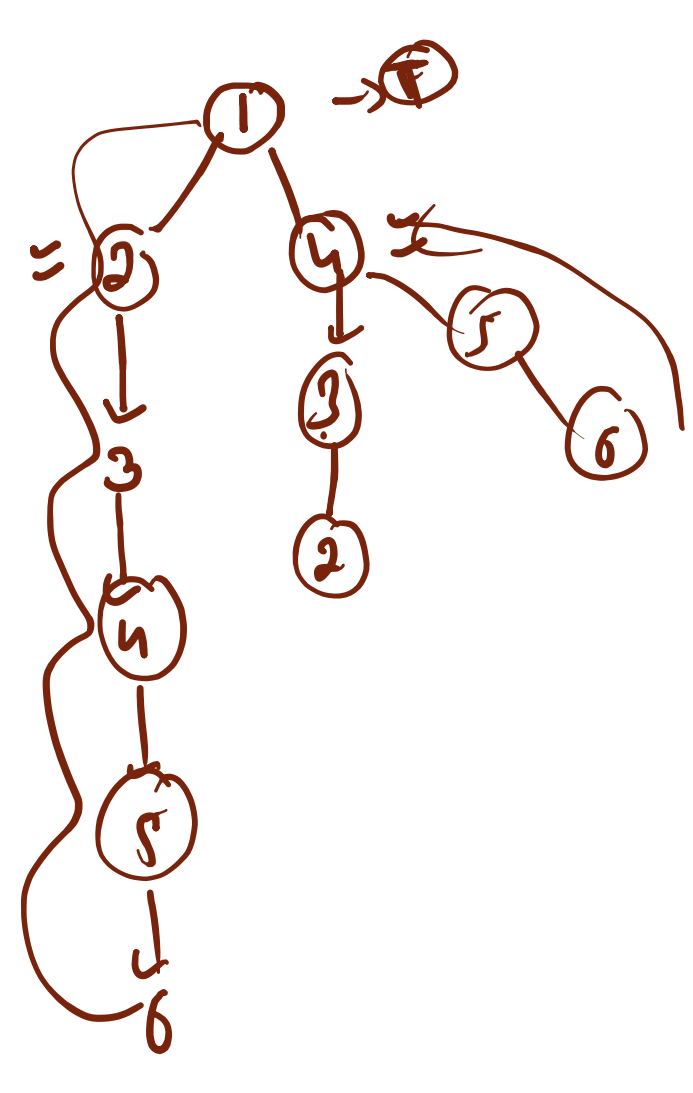
}
map.remove(v)

Index	HashMap <int, int>
1	2K
2	3K
3	4K
4	8K
5	6K
6	7K
7	8K

~11K~

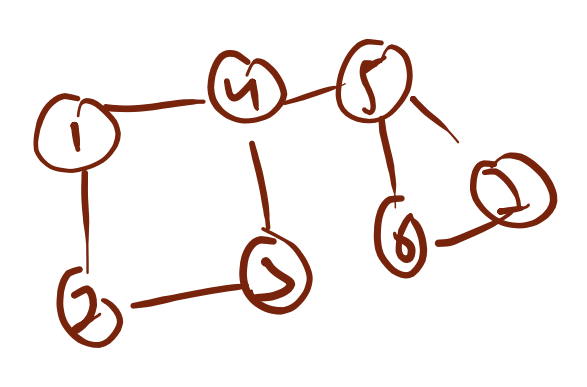

4|5
2|3
2K
1|3
3|7
3K
2|7
4|2
4K
1|5
5|4
3|2
5K
4|1
6|4
7|2
6K
5|4
7|3
7K
6|3
5|2
8K



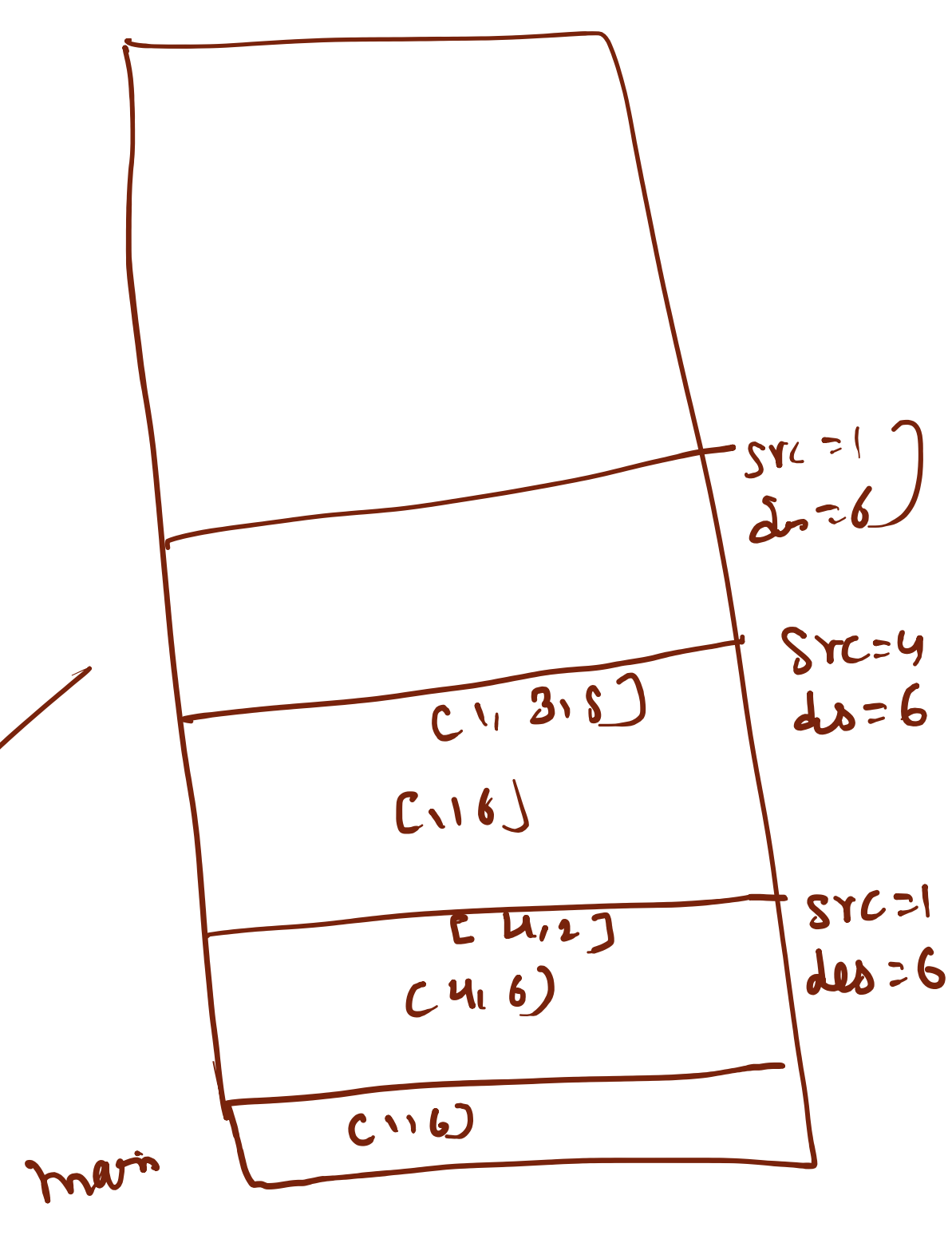

1
F
2
4
3
4
5
6
3
2
5
6

```
public boolean haspath(int src,int des) {
    if(src==des) {
        return true;
    }
    for(int nbrs:map.get(src).keySet()) {
        boolean ans=haspath(nbrs, des);
        if(ans) {
            return ans;
        }
    }
    return false;

}
```

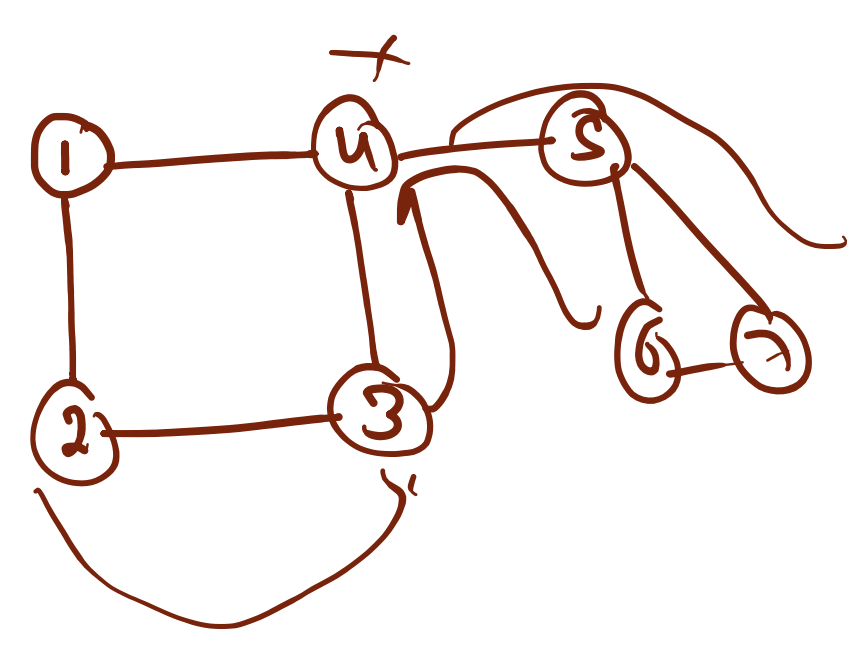


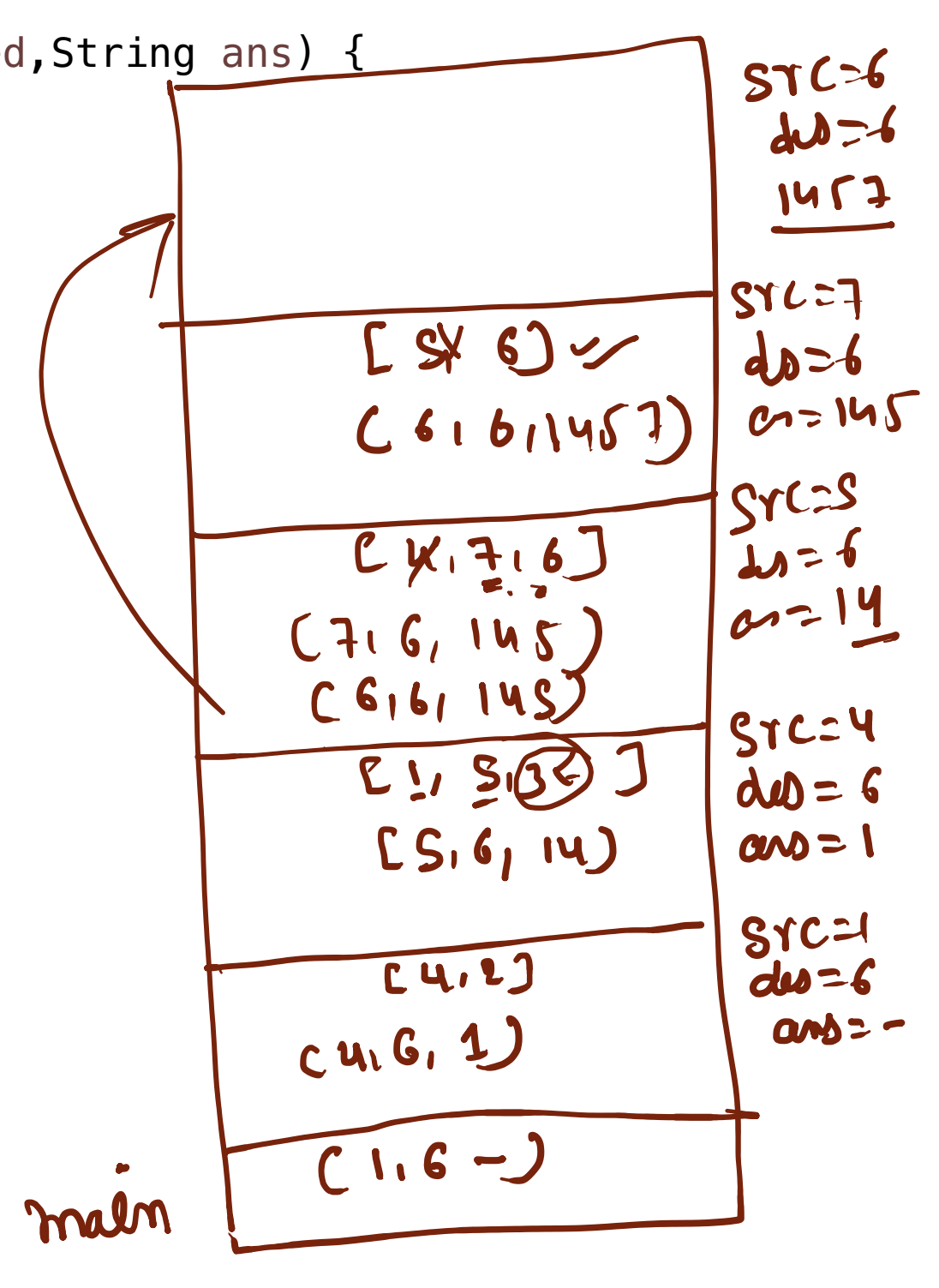
1456
14576
1234576
~~123456~~

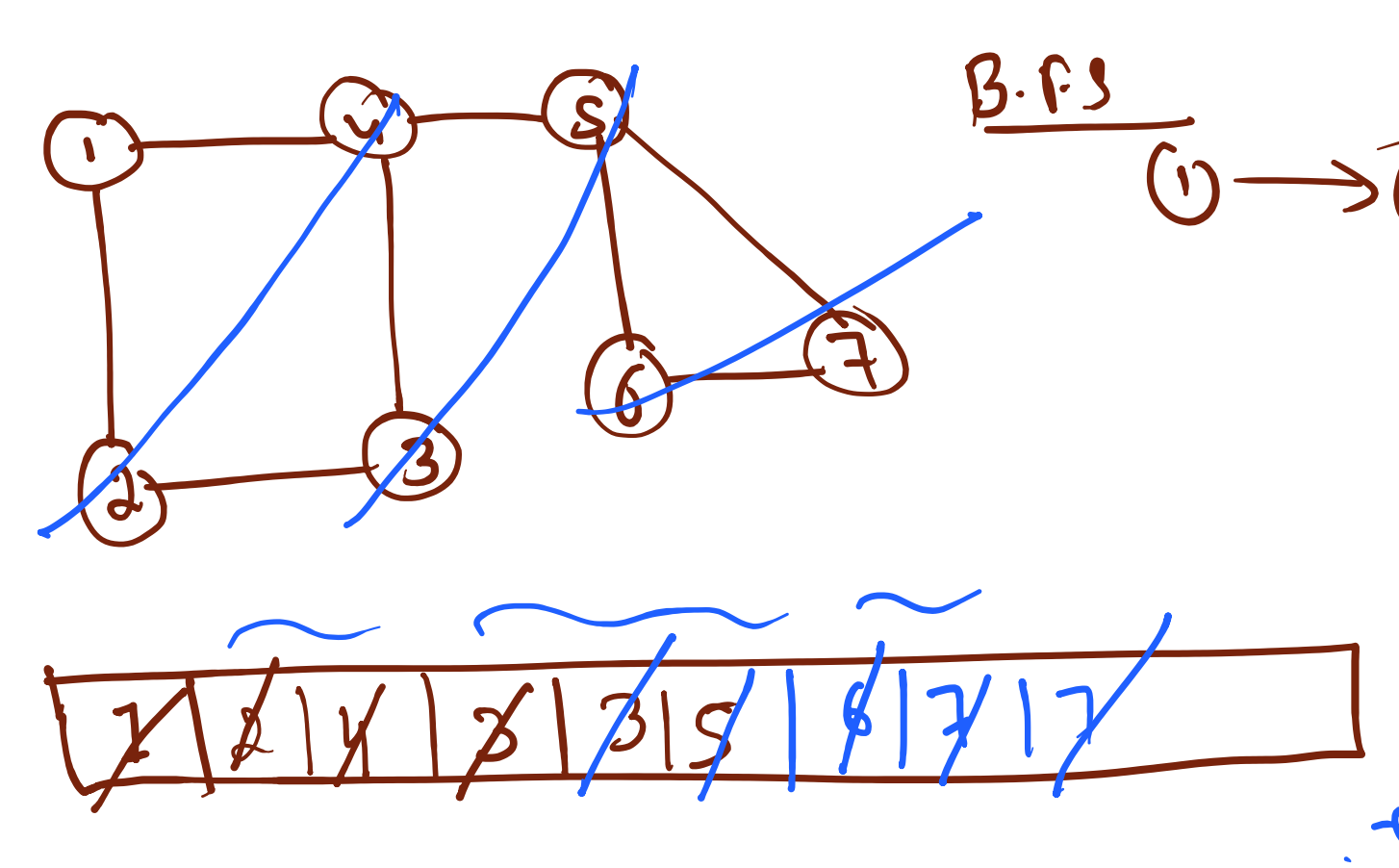

src=1
des=6
src=4
des=6
[1,3,5]
(1,6)
src=1
des=6
[4,2]
(4,6)
(1,6)
main

```
public void printAllpath(int src, int des, HashSet<Integer> visited,String ans) {
    if (src == des) {
        System.out.println(ans);
        return;
    }
    visited.add(src);
    for (int nbrs : map.get(src).keySet()) {
        if (!visited.contains(nbrs)) {
            printAllpath(nbrs, des, visited,ans+src);
        }
    }
}
```

1457




src=6
des=6
1457
[5,6]
(6,6,1457)
src=7
des=6
ans=145
[4,7,6]
(7,6,145)
(6,6,145)
src=5
des=6
ans=14
[1,5,3]
[5,6,14]
src=4
des=6
ans=1
[4,2]
(4,6,1)
src=1
des=6
ans=-
(1,6,-)
main


1
2
3
4
5
6
7
1
2
4
3
5
6
7

1

B.F.S ① → ⑥

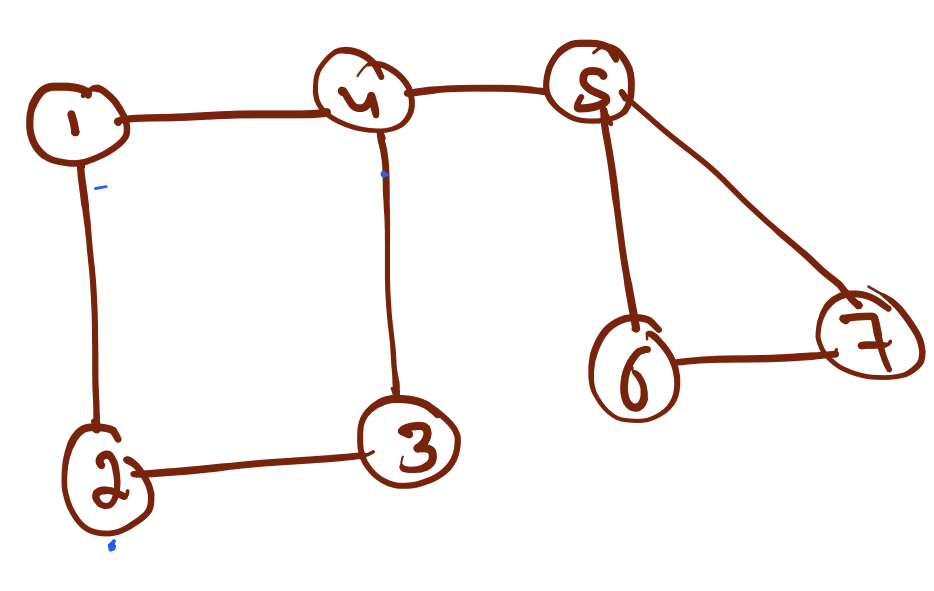
1 2 4
3 5 6 7

for(int nbrs: map.get(v).keySet())

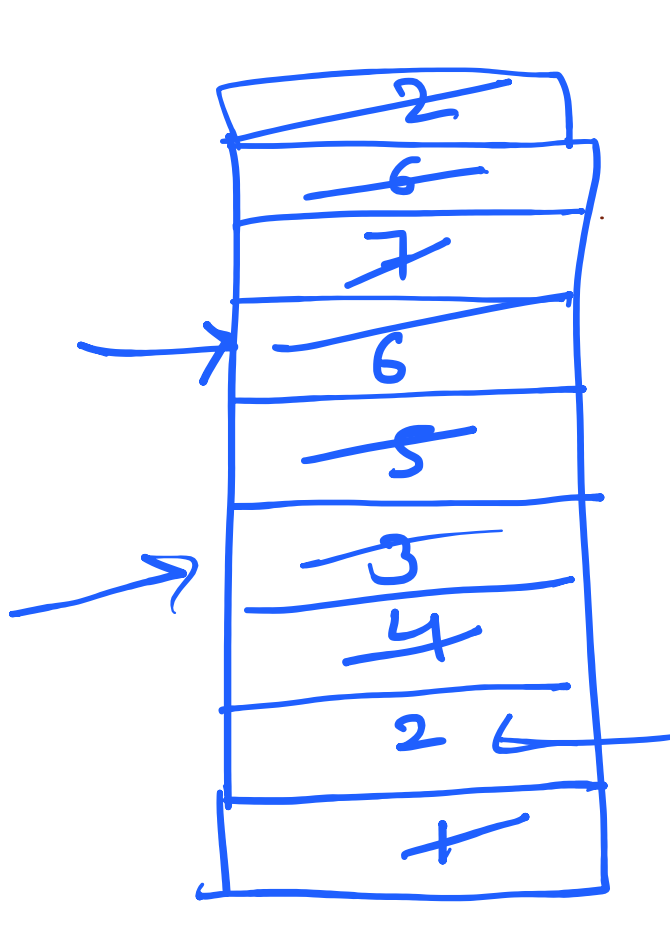
B.F.S ✓ → Tree Level
↓
Queue

D.F.S ✓
↳ Recursion

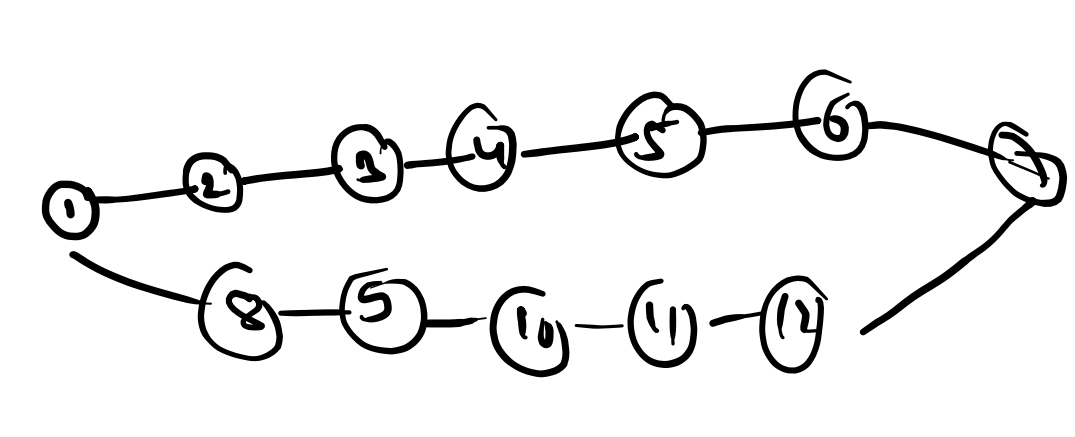
1. remove
2. ignore
3. Mark visited
4. self work
5. Add unvisited



1457
362

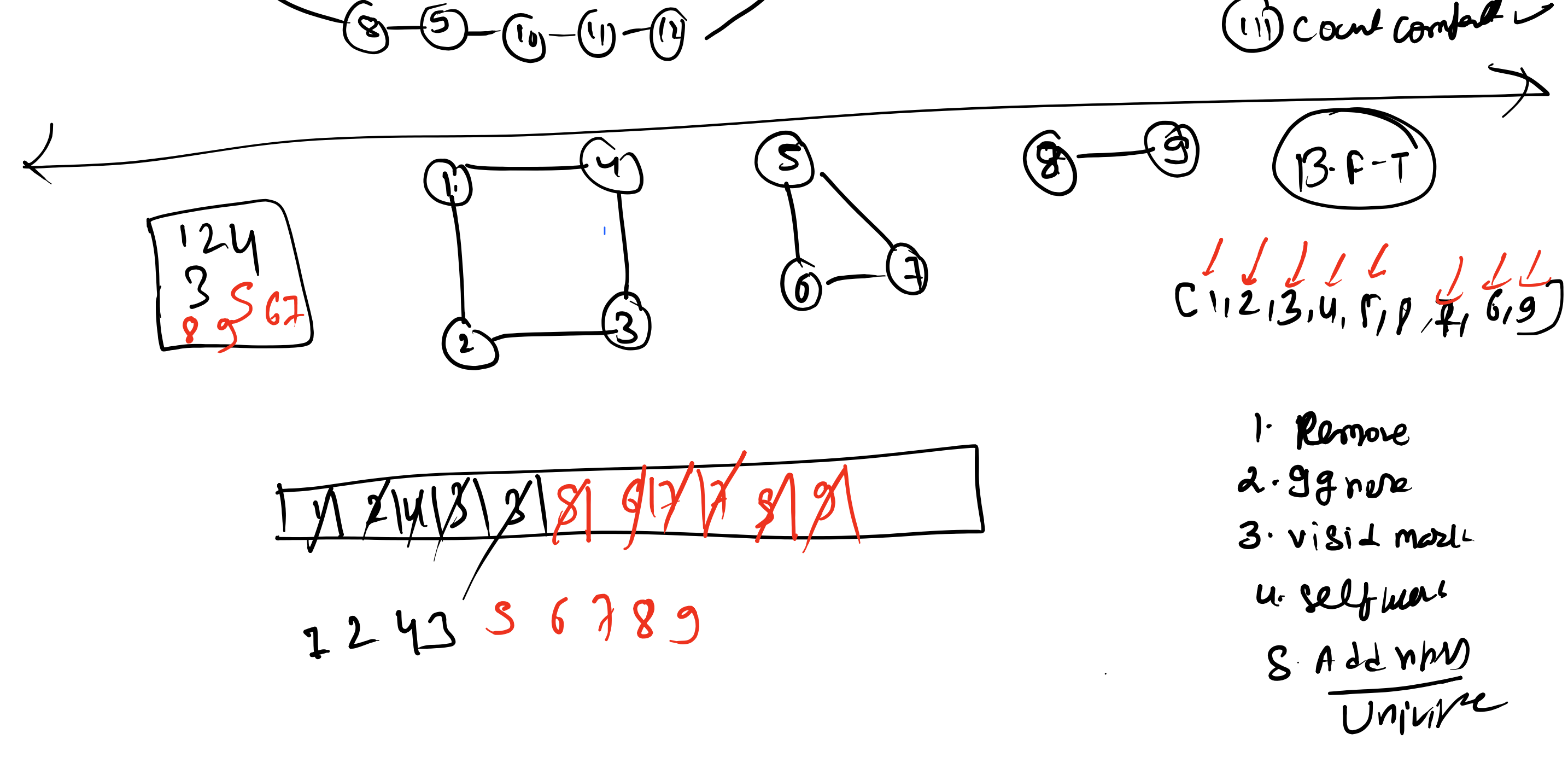


1. remove
2. ignore ×
3. Mark visited
→ 4. self work
5. Add unvisited

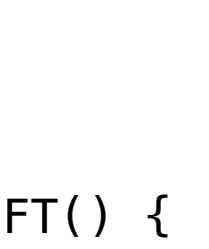

1
2
3
4
5
6
7
8
5
10
11
12

① connected ✓
disconnected ✓
③ count component ✓

124
3 5 67
8 9


1
2
3
4
5
6
7
8
9
B-F-T

[1,2,3,4,5,6,7,6,9]

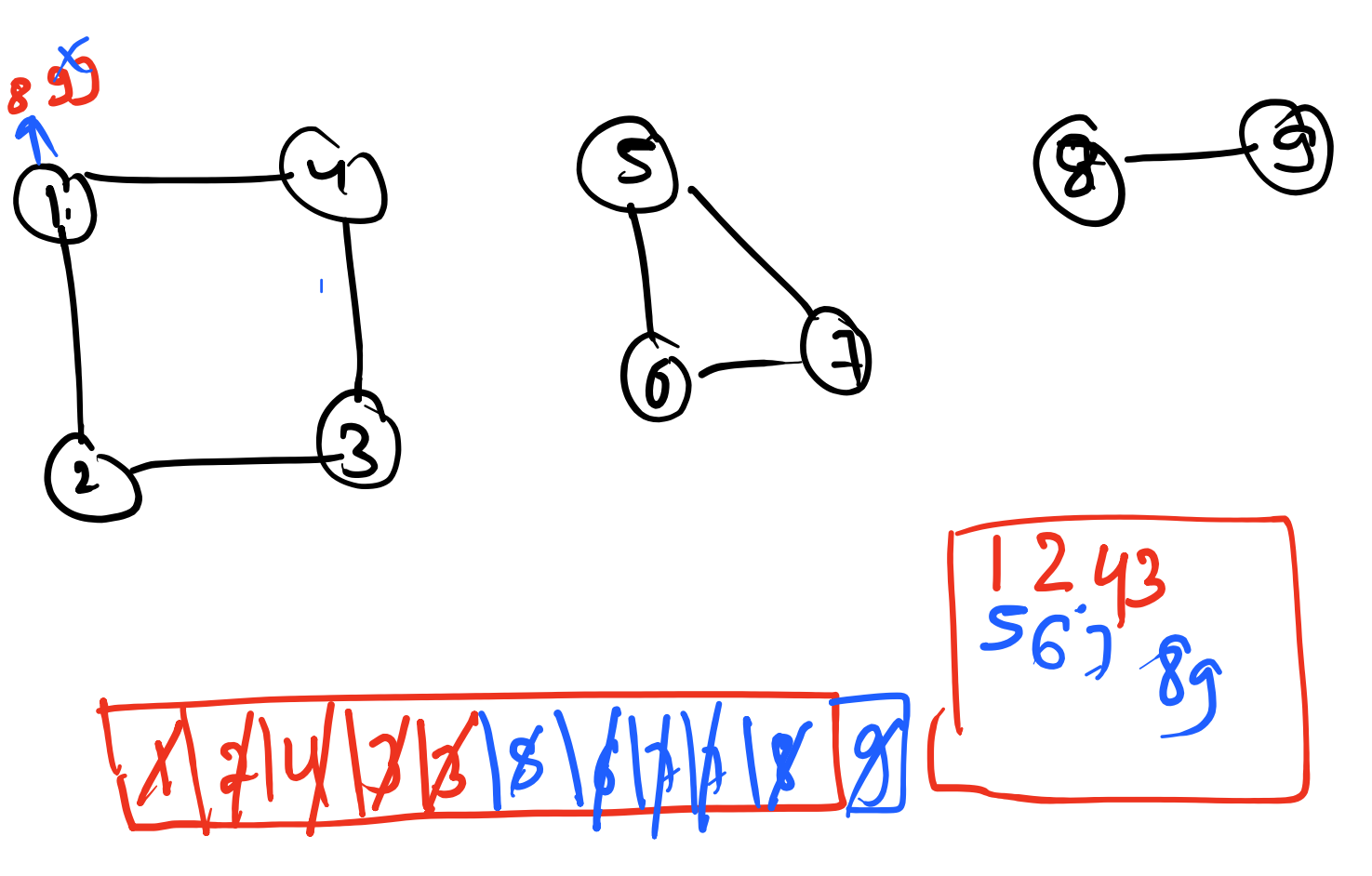

1
2
4
3
3
8
6
7
7
8
9

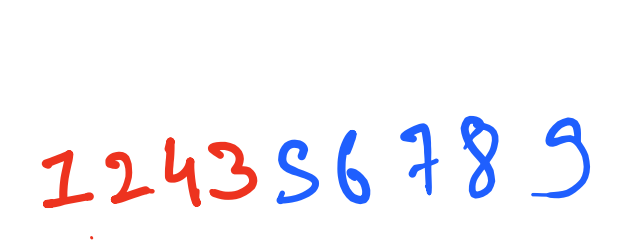
1 2 4 3 5 6 7 8 9

1. Remove
2. ignore
3. visited mark
4. self work
5. Add nbrs Unvisited

```
public void BFT() {
        HashSet<Integer> visited = new HashSet<>();
        Queue<Integer> q = new LinkedList<>();
        for (int src : map.keySet()) {
            if (visited.contains(src)) {
                continue;
            }
            q.add(src);
            while (!q.isEmpty()) {
                // 1. remove
                int v = q.poll();
                // 2.Ignore if Already visited
                if (visited.contains(v)) {
                    continue;
                }
                // 3. marked visted
                visited.add(v);
                // 4. self work
                System.out.print(v + " ");
                // Add unvisited nbsr
                for (int nbrs : map.get(v).keySet()) {
                    if (!visited.contains(nbrs)) {
                        q.add(nbrs);
                    }
                }
            }
        }
        System.out.println();
    }
```

[1,2,3,4,5,6,7,8,9]




1
2
4
3
3
8
6
7
7
8
9

1243
567
89

124356789